

# Software Engineering in the Age of AI (Python-Centric)

## Course Level

Undergraduate

## Course Description

This course introduces modern software engineering principles and practices with a strong emphasis on AI-augmented software development. Students will learn traditional software engineering concepts such as Agile development, requirements engineering, software architecture, testing, DevOps, and maintainability, while also exploring how modern AI tools are transforming software development workflows.

The course uses Python as the primary programming language for demonstrations, laboratory work, and projects. Students will gain hands-on experience building software systems using modern development practices, AI-assisted coding tools, automated testing, and collaborative development workflows.

---



## Course Learning Outcomes

By the end of this course, students will be able to:

1. Explain the fundamental principles of software engineering and software development life cycle (SDLC).
  2. Apply Agile software development practices in team-based projects.
  3. Design and develop backend software systems using Python.
  4. Use AI-assisted development tools responsibly and effectively.
  5. Perform software testing, debugging, and refactoring.
  6. Apply version control and collaborative development workflows.
  7. Design maintainable software architectures.
  8. Implement CI/CD pipelines for software deployment.
  9. Evaluate software quality, security, and ethical considerations in AI-assisted development.
  10. Develop a complete software project using modern engineering practices.
-



## Tools and Technologies

- Python
  - Git & GitHub
  - FastAPI / Flask
  - pytest
  - GitHub Actions
  - AI-assisted development tools (e.g., ChatGPT, GitHub Copilot)
- 



## Recommended Textbooks and Resources

### Primary Textbook

- Engineering Software as a Service  
Armando Fox and David Patterson

### Additional References

- Clean Architecture
  - Refactoring
  - The Pragmatic Programmer
- 



## Weekly Course Schedule

---

### Week 1: Introduction to Modern Software Engineering

#### Topics

- Evolution of software engineering
- Traditional vs AI-augmented development
- Overview of SDLC

- Role of AI in modern software development

## Reading

- Engineering Software as a Service — Chapter 1

## Resources

- <https://martinfowler.com/articles/ai-assisted-programming.html>

## Laboratory

- Python environment setup
  - Git and GitHub introduction
  - Exploring AI coding assistants
- 

# Week 2: Agile Development and SaaS Foundations

## Topics

- Agile methodology
- Scrum framework
- SaaS architecture
- Continuous delivery concepts

## Reading

- Engineering Software as a Service — Chapters 2 and 3

## Resources

- <https://agilemanifesto.org>

## Laboratory

- Building a simple Python web application using Flask
- 

# Week 3: Requirements Engineering with AI Assistance

## Topics

- Software requirements engineering
- Functional and non-functional requirements
- User stories and use cases
- AI-assisted requirement generation

## Reading

- Engineering Software as a Service — Chapter 7

## Resources

- <https://www.promptingguide.ai>

## Laboratory

- Creating software requirements documents
  - Generating user stories using AI tools
- 

# Week 4: Python Backend Engineering

## Topics

- REST APIs
- Modular software design
- Clean code principles
- Backend architecture basics

## Reading

- Engineering Software as a Service — Chapter 4

## Resources

- <https://fastapi.tiangolo.com>

## Laboratory

- Building REST APIs using FastAPI
-

## Week 5: AI-Assisted Coding and Prompt Engineering

### Topics

- AI-assisted software development
- Prompt engineering fundamentals
- Risks and limitations of AI-generated code
- Human verification of AI outputs

### Resources

- <https://www.promptingguide.ai/techniques>
- <https://docs.github.com/en/copilot>

### Laboratory

- Generating and refactoring Python code using AI tools
- 

## Week 6: Software Testing in the Age of AI

### Topics

- Software testing fundamentals
- Unit testing and integration testing
- AI-generated test cases
- Test coverage and quality

### Reading

- Engineering Software as a Service — Chapter 8

### Resources

- <https://docs.pytest.org>

### Laboratory

- Writing automated tests using pytest
-

# Week 7: Version Control and Collaborative Development

## Topics

- Git fundamentals
- Branching and merging
- Pull requests and code review
- AI-assisted collaboration workflows

## Reading

- Engineering Software as a Service — Chapter 10

## Resources

- <https://guides.github.com/introduction/git-handbook/>

## Laboratory

- Team collaboration using GitHub
- 

# Week 8: DevOps and CI/CD

## Topics

- DevOps principles
- Continuous Integration and Continuous Deployment
- Automated workflows
- AI-assisted debugging and monitoring

## Reading

- Engineering Software as a Service — Chapter 12

## Resources

- <https://docs.github.com/en/actions>

## Laboratory

- Building CI/CD pipelines using GitHub Actions

---

## Week 9: Software Architecture and Design

### Topics

- Software architecture fundamentals
- Monolithic vs microservice architectures
- Layered architecture
- Design patterns

### Reading

- Clean Architecture — Chapters 1–6

### Resources

- <https://martinfowler.com/articles/microservices.html>

### Laboratory

- Refactoring applications into layered architecture
- 

## Week 10: Code Quality and Refactoring

### Topics

- Code smells
- Refactoring techniques
- Maintainability
- AI-assisted code improvement

### Reading

- Refactoring — Chapters 1–3

### Resources

- <https://refactoring.guru>

### Laboratory

- Refactoring poorly designed codebases
- 

## **Week 11: Ethics, Security, and Reliability**

### **Topics**

- Ethics in AI-assisted software development
- Security risks in software systems
- Secure coding practices
- Reliability and trustworthiness

### **Resources**

- <https://owasp.org/www-project-top-ten/>
- <https://www.ibm.com/topics/ai-ethics>

### **Laboratory**

- Security analysis and code review exercises
- 

## **Week 12: Final Project Presentation**

### **Topics**

- Final project demonstration
- Project documentation
- Team presentations
- Reflection on AI-assisted software engineering

### **Resources**

- <https://www.writethedocs.org/guide/>

### **Final Project Requirements**

Students must develop a complete Python-based software system incorporating:

- Backend API
- Testing



- Version control
- CI/CD pipeline
- AI-assisted development workflow